

From a 3-Tier App to a Dockerized Deployment

Containerize a React frontend, a Node.js API, and MongoDB, wire them together with Docker Compose, keep the data alive with volumes, and ship the whole thing to an AWS EC2 instance.

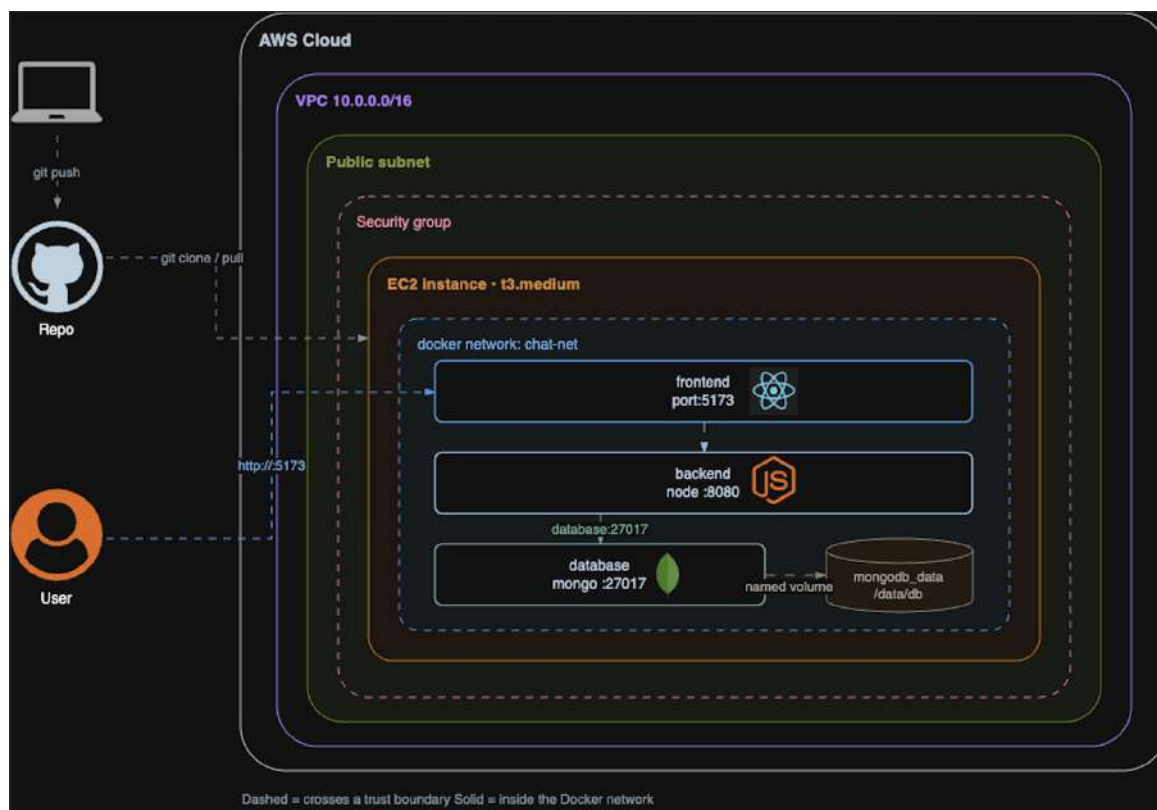
What you will build	What you will understand
A three-container stack running behind one command	Why multi-stage builds cut image size and attack surface
Multi-stage Dockerfiles for the frontend and the API	Why build time and runtime environment variables behave differently
A private Docker network where the API talks to the database by name	Why containers reach each other by service name and not by localhost
A named volume so messages survive a container restart	How to read a connection failure instead of guessing at it
A deployment on EC2 reachable from a browser	Where this setup stops being enough for production

Prerequisites: comfort with the Linux command line, an AWS account, and a full stack app you can already run locally. Everything else is explained as it comes up.

Contents

- 01 The app before Docker
- 02 Containerizing the frontend
- 03 Containerizing the backend
- 04 Environment variables
- 05 Docker Compose
- 06 Building and running the stack
- 07 Debugging like an engineer
- 08 Auth, cookies and websockets
- 09 MongoDB persistence
- 10 Deploying on EC2
- 11 Verifying the deployment
- 12 Where this goes next
- 13 Command reference

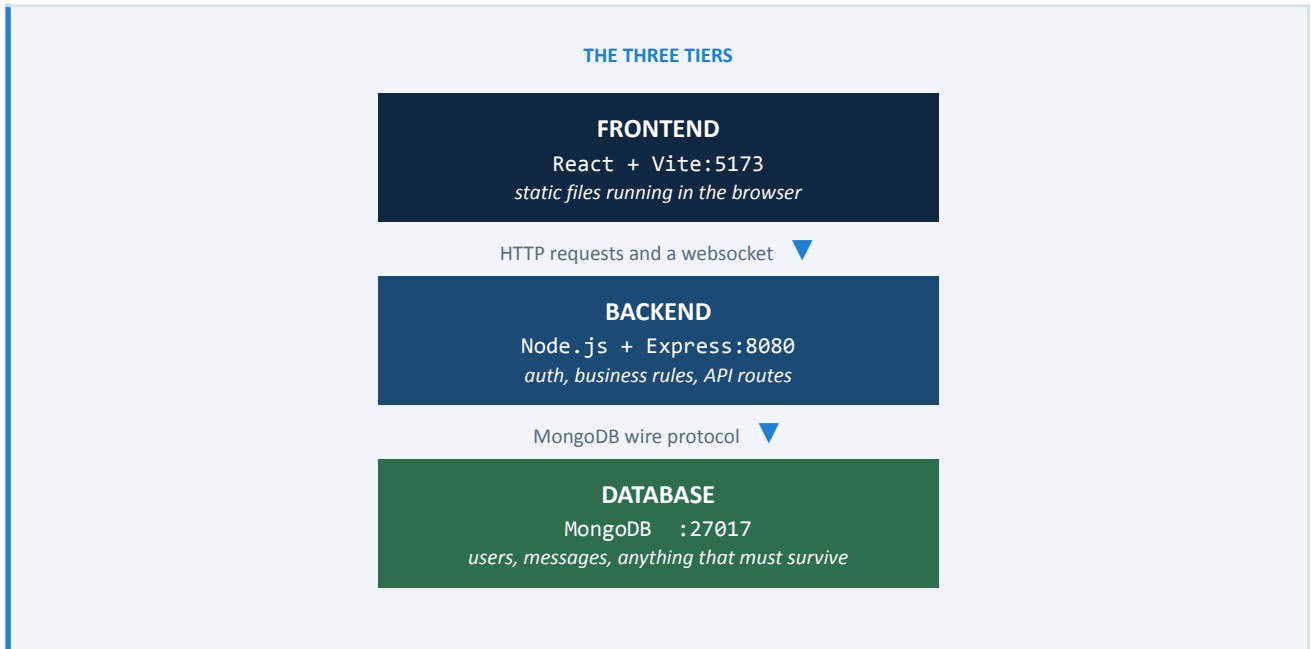
System Design



The app before Docker

You cannot containerize an application you cannot describe. Start here.

The application in this guide is a chat app with three tiers. A React frontend built with Vite, a Node.js and Express API, and MongoDB for storage. Each tier does one job and talks only to the tier below it.



Presentation, application, data. Each tier talks only to the one below it.

Tier	Stack	Port	What it owns
Frontend	React, Vite, Axios, Socket.IO client	5173	UI, routing, calling the API
Backend	Node.js, Express, JWT, Socket.IO	8080	Auth, business rules, database access
Database	MongoDB	27017	Users, messages, everything that must survive

Why this is annoying without Docker

To run this on a fresh machine, you install the right Node version, install MongoDB, start the database service, create an env file, run the API, then run the frontend in a second terminal. Miss one step, and you get an error that has nothing to do with your code. This is where the phrase "works on my machine" is born.

THE SHIFT IN THINKING

Docker does not just run your app. It captures the answer to the question "what does this app need in order to start", writes it down as a file, and makes that answer repeatable on any machine with Docker installed.

Containerizing the frontend

A React app is not a running server. It is a folder of static files. That single fact shapes the whole Dockerfile.

When you run the Vite build, your components, imports, and JSX get bundled into plain HTML, CSS, and JavaScript inside a dist folder. Node is needed to produce those files. Node is not needed to serve them.

That is the case for a multi-stage build. Stage one has the full toolchain and does the work. Stage two takes only the finished output and throws the toolchain away.



Stage one does the work. Stage two keeps the output and throws the toolchain away.

frontend/Dockerfile

FRONTEND/DOCKERFILE

```

# ----- stage 1: build the static bundle -----
FROM dhi.io/node:24-alpine3.23-dev AS builder

WORKDIR /app

# copy manifests first so this layer is cached
# until dependencies actually change
COPY package*.json ./

RUN npm ci

COPY . .

# Vite reads this at BUILD time, so it must be a build arg
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL

RUN npm run build

# ----- stage 2: serve it -----
FROM dhi.io/node:24-alpine3.23 AS runner

WORKDIR /app

COPY --from=builder /app/dist ./dist
COPY server.js .

EXPOSE 5173

CMD ["node", "server.js"]

```

```

ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application/frontend$ ls -a
. .dockerignore README.md index.html package-lock.json postcss.config.js server.js tailwind.config.js
.. Dockerfile eslint.config.js node_modules package.json public src vite.config.js
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application/frontend$ cat Dockerfile
FROM dhi.io/node:24-alpine3.23-dev AS builder

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

ARG VITE_API_URL

ENV VITE_API_URL=$VITE_API_URL

RUN npm run build

FROM dhi.io/node:24-alpine3.23 AS runner

WORKDIR /app

COPY --from=builder /app/dist ./dist
COPY server.js .

EXPOSE 5173

CMD ["node", "server.js"]
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application/frontend$

```

COMMON MISTAKE

Forgetting a .dockerignore. Without it, your local node_modules and .env get copied into the build context, which makes builds slow and can bake secrets into an image.

Both the frontend and the backend need one, and the contents are almost identical.

FRONTEND/.DOCKERIGNORE AND BACKEND/.DOCKERIGNORE

```

node_modules
dist
.git
.env
*.log

```

```

Downloads - ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application/frontend - ssh - (1031.pam.ubuntu@44.211.223.85 ~ 101x34)
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application/frontend$ ls .dockerignore
.dockerignore
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application/frontend$ cat .dockerignore
node_modules
.env
.git
dist
*.log
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application/frontend$

```

Containerizing the backend

The API does need Node at runtime. It does not need your dev dependencies, your tests, or root privileges.

BACKEND/DOCKERFILE

```
# ----- stage 1: production dependencies only -----
FROM dhi.io/node:24-alpine3.23-dev AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

# ----- stage 2: runtime -----
FROM dhi.io/node:24-alpine3.23 AS runner

WORKDIR /app

# Ensures we run as a non-root user for security
USER node

COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package*.json ./
COPY --from=builder /app/src ./src

ENV NODE_ENV=production

EXPOSE 8080

CMD ["node", "src/index.js"]
```

```
ubuntu@19-172-31-86-157:~/FullStack-Chat-Application/Backend$ ls -la
.  ..  .dockerignore  Dockerfile  package-lock.json  package.json  src
ubuntu@19-172-31-86-157:~/FullStack-Chat-Application/Backend$ cat Dockerfile
FROM dhi.io/node:24-alpine3.23-dev AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

FROM dhi.io/node:24-alpine3.23 AS runner

WORKDIR /app

# Ensures we run as a non-root user for security
USER node

COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package*.json ./
COPY --from=builder /app/src ./src

ENV NODE_ENV=production
EXPOSE 8080

CMD ["node", "src/index.js"]

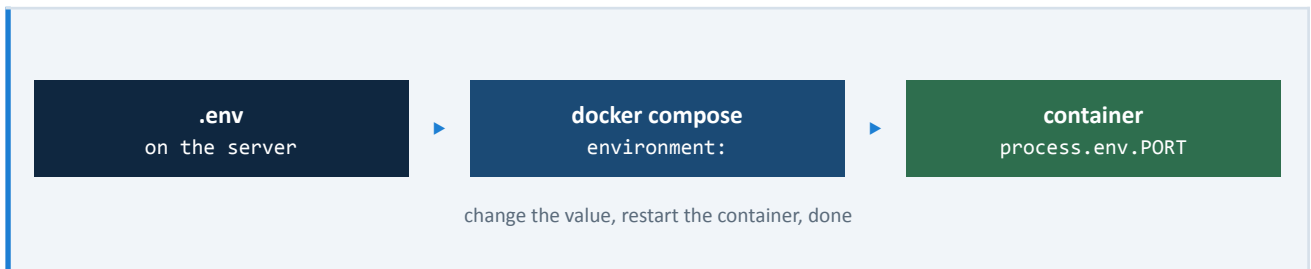
ubuntu@19-172-31-86-157:~/FullStack-Chat-Application/Backend$
```

Environment variables, and the trap inside them

Two variables can sit in the same `.env` file and behave completely differently. This is the single most common reason a first deployment loads but does not work.

Runtime variables: the backend

Node reads `process.env` when the process starts. Change the value, restart the container, and the new value is picked up. Nothing has to be rebuilt.



Runtime variables are read when the process starts.

Build time variables: the frontend

Vite does not read environment variables in the browser, because a browser has no environment. During the build, it finds every `import.meta.env.VITE_API_URL` and replaces it with a literal string inside the bundle. After that, the value is frozen into JavaScript, and public: it ships to every visitor and is readable in devtools, so secrets never belong behind a `VITE_` prefix.



Build time variables are baked into the JavaScript your users download. A restart will not change them.

Variable	Lives where	Changing it needs
<code>PORT</code>	Backend container env	A restart
<code>MONGODB_URI</code>	Backend container env	A restart
<code>JWT_SECRET</code>	Backend container env	A restart
<code>VITE_API_URL</code>	Baked into the frontend bundle	A rebuild of the image

THE TRAP

If you change VITE_API_URL and only run docker compose restart, nothing happens. The old value is still sitting in the built JavaScript. You have to rebuild the frontend image.

Fix: `docker compose build frontend && docker compose up -d frontend`

```
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ ll
total 52
drwxrwxr-x 5 ubuntu ubuntu 4096 Sep  2 15:51 ./
drwxr-x--- 8 ubuntu ubuntu 4096 Sep  2 15:53 ../
-rw-rw-r-- 1 ubuntu ubuntu  133 Sep  2 13:29 .env
-rw-rw-r-- 1 ubuntu ubuntu  158 Sep  2 15:48 .env.example
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep  2 15:57 .git/
-rw-rw-r-- 1 ubuntu ubuntu  341 Sep  2 15:51 .gitignore
-rw-rw-r-- 1 ubuntu ubuntu 2603 Aug 31 08:28 Readme.md
drwxrwxr-x 3 ubuntu ubuntu 4096 Sep  2 15:47 backend/
-rw-rw-r-- 1 ubuntu ubuntu  775 Sep  2 13:12 docker-compose.yml
drwxrwxr-x 5 ubuntu ubuntu 4096 Sep  2 15:47 frontend/
-rw-rw-r-- 1 ubuntu ubuntu 5850 Aug 31 08:28 package-lock.json
-rw-rw-r-- 1 ubuntu ubuntu  364 Aug 31 08:28 package.json
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ cat .env.example
EC2_PUBLIC_IP=your-ec2-public-ip
JWT_SECRET=your-jwt-secret
MONGO_INITDB_ROOT_USERNAME=your-mongodb-username
MONGO_INITDB_ROOT_PASSWORD=your-mongodb-password
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$
```


Docker Compose: connecting the three tiers

Three Dockerfiles give you three isolated containers. Compose is what turns them into one application.

DOCKER-COMPOSE.YML

```
services:
  frontend:
    build:
      context: ./frontend
      args:
        VITE_API_URL: http://${EC2_PUBLIC_IP}:8080/api
    ports:
      - 5173:5173

  backend:
    build:
      context: ./backend
    ports:
      - 8080:8080
    networks:
      - chatapp-net
    environment:
      PORT: 8080
      JWT_SECRET: "${JWT_SECRET}"
      MONGODB_URI:
        "mongodb://${MONGO_INITDB_ROOT_USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@database:27017/chatapp?authSource=admin"

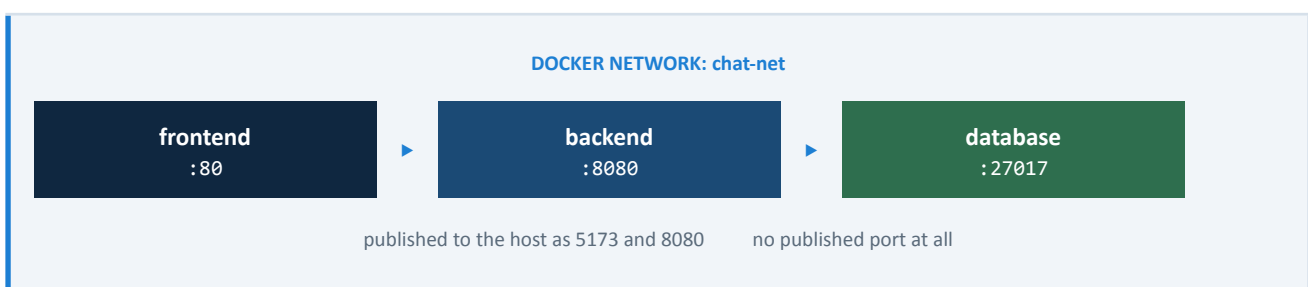
  database:
    image: mongo:7.0-jammy
    environment:
      MONGO_INITDB_ROOT_USERNAME: "${MONGO_INITDB_ROOT_USERNAME}"
      MONGO_INITDB_ROOT_PASSWORD: "${MONGO_INITDB_ROOT_PASSWORD}"
    volumes:
      - mongodb:/data/db
    networks:
      - chatapp-net

networks:
  chatapp-net:

volumes:
  mongodb:
```

The networking idea worth memorising

Compose creates a user-defined bridge network and puts every service on it. Inside that network, each service name becomes a DNS name. The backend reaches the database at database:27017 because that is the service name in the file.



Published ports are doors to the outside world. The database does not get one.

WHY LOCALHOST FAILS INSIDE A CONTAINER

Every container has its own network namespace. localhost inside the backend container means the backend container itself, not the host and not MongoDB. Point the connection string at the service name and Docker resolves it for you.

```
# wrong inside Compose
MONGODB_URI=mongodb://localhost:27017/chatdb

# right
MONGODB_URI=mongodb://database:27017/chatdb
```

```
ubuntu@ip-172-31-86-167:~/Fullstack-Chat-Application$ cat docker-compose.yml
services:
  frontend:
    build:
      context: ./frontend
      args:
        VOTE_API_URL: https://ELEC2_PUBLIC_IP:8080/api
    ports:
      - 5173:5173

  backend:
    build:
      context: ./backend
    ports:
      - 8080:8080
    networks:
      - chatapp-net
    environment:
      PORT: 8080
      JWT_SECRET: "${JWT_SECRET}"
      MONGODB_URI: "mongodb://${MONGO_INITDB_ROOT_USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@database:27017/chatapp?authSource=admin"

  database:
    image: mongo:7.0-jammy
    environment:
      MONGO_INITDB_ROOT_USERNAME: "${MONGO_INITDB_ROOT_USERNAME}"
      MONGO_INITDB_ROOT_PASSWORD: "${MONGO_INITDB_ROOT_PASSWORD}"
    volumes:
      - mongodb:/data/db
    networks:
      - chatapp-net

networks:
  chatapp-net:

volumes:
  mongodb:
```

Building and running the stack

Four commands, and a clear idea of what each one changes.

```
# build images from the Dockerfiles
docker compose build

# start everything in the background
docker compose up -d

# what is actually running
docker compose ps

# follow the logs of one service
docker compose logs -f backend
```

Command	What it does	When you need it
<code>docker compose build</code>	Runs each Dockerfile and produces images	After a code change or a new build arg
<code>docker compose up -d</code>	Creates network, volumes and containers, then starts them	Every time you deploy
<code>docker compose restart backend</code>	Restarts the process, same image	After changing a runtime env value
<code>docker compose down</code>	Stops and removes containers and the network	Cleaning up between runs
<code>docker compose down -v</code>	All of the above, plus deletes named volumes	Almost never, see section 09

```
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ ls
Readme.md backend docker-compose.yml frontend package-lock.json package.json
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ docker compose up -d
[+] Running 11/11
 ✓ database Pulled 11.2s
 ✓ 164558a2a8fa Pull complete 2.8s
 ✓ d544298cabd5 Pull complete 2.7s
 ✓ 77214491514e Pull complete 0.2s
 ✓ 9dad1a8753e0 Pull complete 0.2s
 ✓ 6bcb0cd9f68c Pull complete 0.1s
 ✓ 4afa172ff42c Pull complete 10.7s
 ✓ 586ba587042d Pull complete 3.0s
 ✓ ad1d6026847d Pull complete 0.2s
 ✓ a2b49326e024 Download complete 0.1s
 ✓ b6bc1923d416 Download complete 0.0s
WARN[0011] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 39.7s (29/29) FINISHED docker:default
=> [frontend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 365B 0.0s
=> [backend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 470B 0.0s
=> [backend internal] load metadata for dhi.io/node:24-alpine3.23 1.2s
=> [frontend internal] load metadata for dhi.io/node:24-alpine3.23-dev 1.3s
=> [backend auth] node:pull token for dhi.io 0.0s
=> [backend internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [frontend internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [backend builder 1/6] FROM dhi.io/node:24-alpine3.23-dev@sha256:609216ef29887f388b3be53ef980b33685ebbbbf36940 7.1s
=> => resolve dhi.io/node:24-alpine3.23-dev@sha256:609216ef29887f388b3be53ef980b33685ebbbbf36940cdd5462245230f28 0.1s
```

```

=> [frontend] installing provenance for metadata file 0.6s
[*] Waiting 7/7
✓ backend Built 0.8s
✓ frontend Built 0.8s
✓ Network fullstack-chat-application_default Created 0.3s
✓ Network fullstack-chat-application_chatapp-net Created 0.1s
✓ Container fullstack-chat-application-backend-1 Started 0.5s
✓ Container fullstack-chat-application-database-1 Started 0.5s
✓ Container fullstack-chat-application-frontend-1 Started 0.7s
shuntu@ip-172-31-85-157:~/Fullstack-Chat-Application$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
b8c4c8819a0c   mongo:7.0-jammy                    "docker-entrypoint.s..." 18 seconds ago Up 18 seconds 27017/tcp
4843f2884e7c   fullstack-chat-application-database-1 "node src/index.js"      18 seconds ago Up 18 seconds 4.9.0.0:4
808->8080/tcp, [::]:8080->8080/tcp fullstack-chat-application-backend-1 "node src/index.js"      18 seconds ago Up 18 seconds 4.9.0.0:4
80c19c83e8ab   fullstack-chat-application-backend-1 "node server.js"         18 seconds ago Up 18 seconds 4.9.0.0:4
173->5173/tcp, [::]:5173->5173/tcp fullstack-chat-application-frontend-1
shuntu@ip-172-31-85-157:~/Fullstack-Chat-Application$

```

Debugging like an engineer

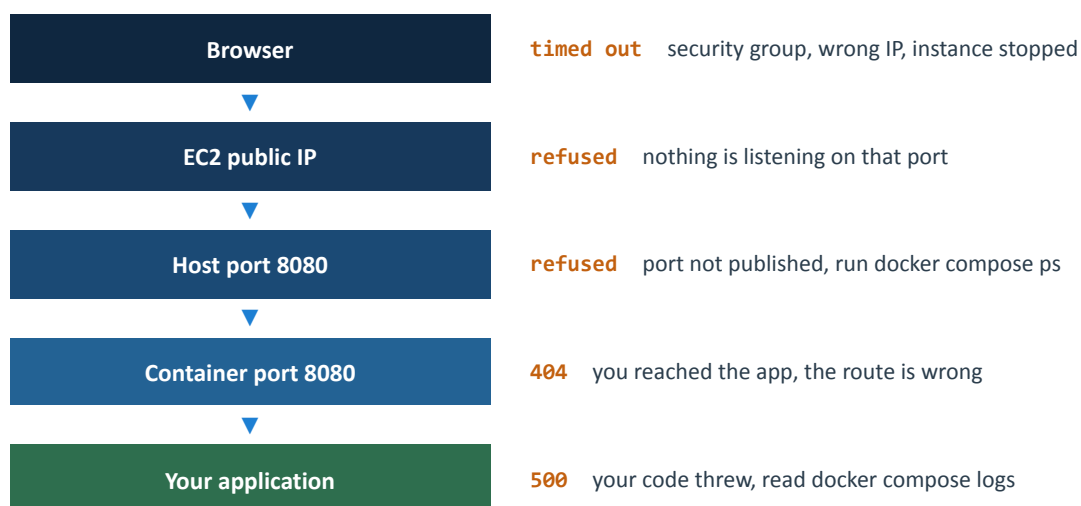
Most deployment failures are not mysterious. The error message tells you exactly how far the request got, if you know how to read it.

Three errors that look similar and mean different things

What you see	How far the request got	Where to look
<code>ERR_CONNECTION_TIMED_OUT</code>	Never received an answer at all	EC2 security group, wrong public IP, instance stopped
<code>ERR_CONNECTION_REFUSED</code>	Reached the machine, nothing was listening on that port	Container not running, port not published, app crashed on boot
<code>404 Not Found</code> from Express	Reached your application, the route does not exist	Missing <code>/api</code> prefix, wrong path, router not mounted
<code>500</code> from the API	Your code ran and threw	Container logs, database connection, missing env var
CORS error in the browser	The API answered, the browser rejected the answer	CORS origin and credentials config

That third row is worth sitting with. When `curl http://localhost:8080` returns the Express 404 page, that is good news. It means the container is up, the port mapping works, and Node is serving. You are no longer debugging infrastructure, you are debugging a route.

Walk the path in order



Never jump ahead. Check each hop before blaming the next one.

The commands that answer each question

```
# are the containers up, and which ports are mapped?
docker compose ps

# is anything listening on the host port?
sudo ss -lntp | grep 8080

# does the app answer from the host itself?
curl -i http://localhost:8080/api/auth/check

# what did the app say while starting?
docker compose logs --tail=50 backend

# did the container get the env vars you think it did?
docker compose exec backend printenv | grep -E "NODE_ENV|MONGODB_URI|JWT"

# can the backend resolve the database by name?
docker compose exec backend ping -c 2 database
```

TWO SILENT TIME WASTERS

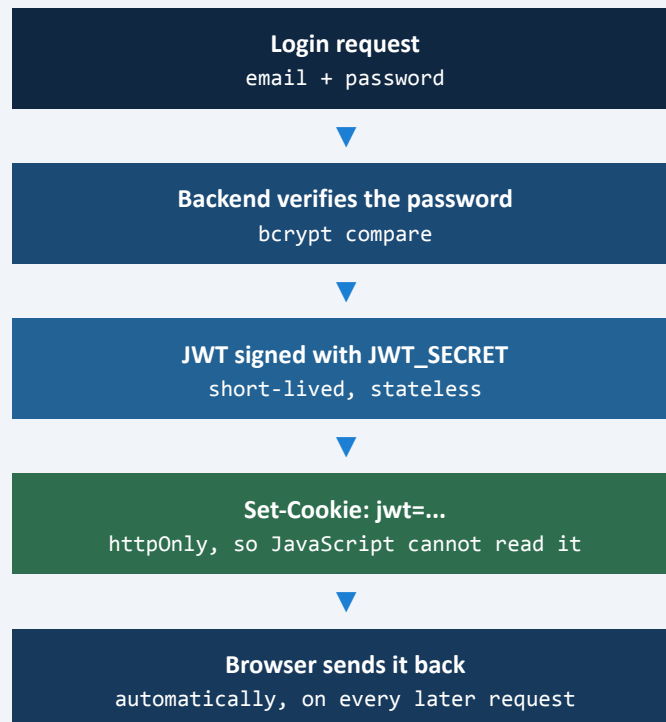
A `grep` that returns nothing is not proof that the code is missing. Check `pwd` first, since running `grep` from your home directory instead of the repo will always come back empty.

AND THE SECOND ONE

`docker exec` with a guessed container name fails quietly in a way that looks like an empty result. Get the real names from `docker compose ps`, or address services through Compose with `docker compose exec backend` so you never have to guess.

Auth, cookies and websockets

The symptom: signup works, login works, and then the app behaves as if nobody is logged in and no other users exist. This is almost always an origin problem, not an auth problem.



No token juggling in React. The browser does the carrying.

Why httpOnly is the right default

- The cookie is invisible to JavaScript, so a cross site scripting bug cannot read the token.
- The browser attaches it for you, so no token juggling in React state or localStorage.
- You can expire it server side by overwriting the cookie on logout.

The two settings that decide whether it works at all

```
// backend, when issuing the cookie
res.cookie("jwt", token, {
  httpOnly: true,
  sameSite: "strict", // strict blocks cross origin sends
  secure: false,      // true requires HTTPS
  maxAge: 7 * 24 * 60 * 60 * 1000
});
```

```
// frontend, axios must be told to send cookies
axios.defaults.withCredentials = true;
```

```
// backend CORS, a wildcard is not allowed with credentials
app.use(cors({
  origin: process.env.CLIENT_URL,    // e.g. http://13.234.x.x:5173
  credentials: true
}));
```

THE HIDDEN CAUSE

Serving the app on port 5173 and the API on port 8080 makes them different origins. With `sameSite: "strict"` the browser quietly drops the cookie, and every following request looks anonymous. You get no error, just an empty app.

The cleaner fix: put them on the same origin

Instead of loosening cookie rules, proxy the API through the same nginx that serves the frontend. The browser then sees one origin, cookies are same site by definition, and CORS stops being a concern.

FRONTEND/NGINX.CONF, WITH THE API PROXIED

```
server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;

  location / {
    try_files $uri $uri/ /index.html;
  }

  location /api/ {
    proxy_pass http://backend:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;

    # required for Socket.IO to upgrade to a websocket
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
  }
}
```

With this in place `VITE_API_URL` becomes `/api`, the backend port no longer needs to be published to the internet, and the online users list starts filling in because the websocket upgrade is allowed through.

IF TWO LOGGED IN USERS STILL CANNOT SEE EACH OTHER

Check the websocket, not the database. An empty `onlineUsers` array with a working REST API is the signature of a socket connection that was blocked or never upgraded. The browser network tab will show the `socket.io` request failing while ordinary requests succeed.

MongoDB persistence, and the flag that erases it

A container filesystem is disposable by design. If you do nothing, removing the database container removes every message in it.

WITHOUT A VOLUME

MongoDB writes to `/data/db` inside the container.
That path is part of the container filesystem.
`docker compose down` removes the container.
Every user and every message goes with it.

WITH A NAMED VOLUME

MongoDB still writes to `/data/db`.
That path is mapped to a Docker managed volume.
The volume lives on the host, outside the container.
Containers come and go, the data stays.

THE PART OF DOCKER-COMPOSE.YML THAT DOES THE WORK

```
services:
  database:
    image: mongo:7
    volumes:
      - mongodb_data:/data/db

volumes:
  mongodb_data:
```

Everything on the left of the colon is the volume name managed by Docker. Everything on the right is the path inside the container. MongoDB writes to `/data/db`, so that is the path that has to be mapped.

THE DIFFERENCE WORTH TATTOOING SOMEWHERE

`docker compose down` removes containers and the network, then recreates them on the next up. Your data survives, because the volume is a separate object with its own lifecycle.

AND ITS DANGEROUS SIBLING

`docker compose down -v` also deletes the named volumes. Every user, every message, gone with no confirmation prompt. It is the right command when you want a clean slate, and a very wrong one on anything you care about.

Proving it, rather than claiming it

```
# the volume exists and is managed by Docker
docker volume ls

# where it lives on the host, and which container uses it
docker volume inspect chatapp_mongodb_data

# the actual test: send a message, then
docker compose down
docker compose up -d
# log back in, the message is still there
```

Check	What it actually proves
<code>docker volume ls</code>	The volume exists as an object Docker manages, separate from any container
<code>docker volume inspect</code>	Where the data sits on the host filesystem, and which container has it mounted
<code>down then up -d</code>	The containers were destroyed and rebuilt, and the data came back with them

A VOLUME IS NOT A BACKUP

The volume protects you from container churn. It does not protect you from a deleted instance, a corrupted disk or a bad migration that writes over your data. Those need a real backup, which on this setup means a scheduled mongodump copied somewhere off the instance.

```

chris@chris-172-31-86-157:~/Fullstack-Chat-Application$ docker inspect fullstack-chat-application-database-1 --format '{{.
  son .Mounts}}'
[{"Type":"volume","Name":"c91c0c4227369fbb68bab6ca99dc665832db86442e551485f576a8871ac98","Source":"/var/lib/docker/v
  olumes/c91c0c4227369fbb68bab6ca99dc665832db86442e551485f576a8871ac98/_data","Destination":"/data/configdb","Driver":"
  local","Mode":"","RW":true,"Propagation":""},{Type":"volume","Name":"fullstack-chat-application-mongod","Source":"/var
  lib/docker/volumes/fullstack-chat-application-mongod/_data","Destination":"/data/ds","Driver":"local","Mode":"rw","RW":
  true,"Propagation":""}]
chris@chris-172-31-86-157:~/Fullstack-Chat-Application$ docker volume ls
DRIVER      VOLUME NAME
local       4ee6f6cd33e3b4d87b71636f970628d4372b2c94cb7b1c048196eb17b787808ca
local       4ed42a7fbae5c86284874841ae7ff12c9bac3c415a426f512218c48dfef8ee
local       92d6a7834eb3568ee655e991ab7be18e27f68f73f67b6a76e3c6bd1fb688f7f
local       9482819b3f98e7d11c8842a35f66cd131be1b1eb2e946f1fb16c873894346
local       c91c0c4227369fbb68bab6ca99dc665832db86442e551485f576a8871ac98
local       fullstack-chat-application-mongod
chris@chris-172-31-86-157:~/Fullstack-Chat-Application$

```

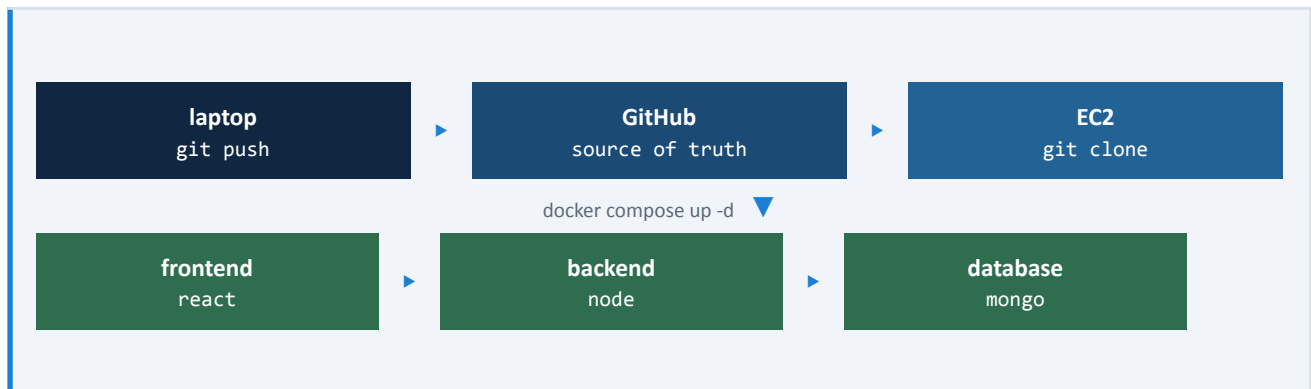
```

chris@chris-172-31-86-157:~/Fullstack-Chat-Application$ docker volume ls
DRIVER      VOLUME NAME
local       4ee6f6cd33e3b4d87b71636f970628d4372b2c94cb7b1c048196eb17b787808ca
local       4ed42a7fbae5c86284874841ae7ff12c9bac3c415a426f512218c48dfef8ee
local       92d6a7834eb3568ee655e991ab7be18e27f68f73f67b6a76e3c6bd1fb688f7f
local       9482819b3f98e7d11c8842a35f66cd131be1b1eb2e946f1fb16c873894346
local       dc468bffb0ff23ae73404a9276efb6059697428a87f7f9e25378b4492947dcab
local       fullstack-chat-application-mongod
chris@chris-172-31-86-157:~/Fullstack-Chat-Application$ docker compose down
[+] Running 3/3
  ✓ Container fullstack-chat-application-frontend-1  Removed      18.1s
  ✓ Container fullstack-chat-application-backend-1   Removed      19.1s
  ✓ Container fullstack-chat-application-database-1  Removed       9.1s
  ✓ Network fullstack-chat-application_default       Removed       8.1s
  ✓ Network fullstack-chat-application_chatapp-net   Removed       8.1s
chris@chris-172-31-86-157:~/Fullstack-Chat-Application$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED   STATUS    PORTS   NAMES
chris@chris-172-31-86-157:~/Fullstack-Chat-Application$ docker volume ls
DRIVER      VOLUME NAME
local       4ee6f6cd33e3b4d87b71636f970628d4372b2c94cb7b1c048196eb17b787808ca
local       4ed42a7fbae5c86284874841ae7ff12c9bac3c415a426f512218c48dfef8ee
local       92d6a7834eb3568ee655e991ab7be18e27f68f73f67b6a76e3c6bd1fb688f7f
local       9482819b3f98e7d11c8842a35f66cd131be1b1eb2e946f1fb16c873894346
local       dc468bffb0ff23ae73404a9276efb6059697428a87f7f9e25378b4492947dcab
local       fullstack-chat-application-mongod
chris@chris-172-31-86-157:~/Fullstack-Chat-Application$ docker compose up -d
[+] Running 3/3
  ✓ Network fullstack-chat-application_chatapp-net   Created       8.1s
  ✓ Network fullstack-chat-application_default       Created       8.1s
  ✓ Container fullstack-chat-application-frontend-1  Started       8.8s
  ✓ Container fullstack-chat-application-backend-1   Started       8.6s
  ✓ Container fullstack-chat-application-database-1  Started       9.6s
chris@chris-172-31-86-157:~/Fullstack-Chat-Application$

```

Deploying on EC2

The local setup and the server setup are the same setup. That is the whole point of the exercise.



The same three files run the stack on your laptop and on the server.

Instance and security group

Setting	Value	Reasoning
Image	Ubuntu 22.04 or 24.04 LTS	Well documented, current Docker packages
Type	t3.medium	A t2.micro often runs out of memory during the Vite build
Storage	16 GB or more	Images, layers, and the Mongo volume add up fast
Port 22	Your IP only	SSH open to the world is asking for trouble
Port 5173	Anywhere	The frontend, or use 80 for a cleaner URL
Port 8080	Anywhere, or closed	Not needed at all if you proxy the API through nginx

sg-0b3bcb922d1eb3e31 - launch-wizard-11

Details

- Security group name: launch-wizard-11
- Security group ID: sg-0b3bcb922d1eb3e31
- Description: launch-wizard-11 created 2026-08-3 1T08:26:25.717Z
- VPC ID: vpc-c09309f8b416deb9d2
- Owner: 252366102382
- Inbound rules count: 4 Permission entries
- Outbound rules count: 1 Permission entry

Inbound rules (4)

Name	Security group rule ID	IP version	Type	Protocol	Port range
SSH	sgr-00d5b46eae612dd8c	IPv4	SSH	TCP	22
Custom TCP	sgr-03ccda8cfd8b3d430	IPv4	Custom TCP	TCP	5173
HTTP	sgr-0e2157a63c0770e18	IPv4	HTTP	TCP	80
Custom TCP	sgr-0b1a1f0ac58868df7	IPv4	Custom TCP	TCP	8080

Setting up the instance

```
# 1. connect
ssh -i chat-key.pem ubuntu@<EC2_PUBLIC_IP>

# 2. install Docker Engine and the Compose plugin
sudo apt update && sudo apt upgrade -y
curl -fsSL https://get.docker.com | sudo sh

# 3. run docker without sudo, then reconnect for it to apply
sudo usermod -aG docker $USER
exit

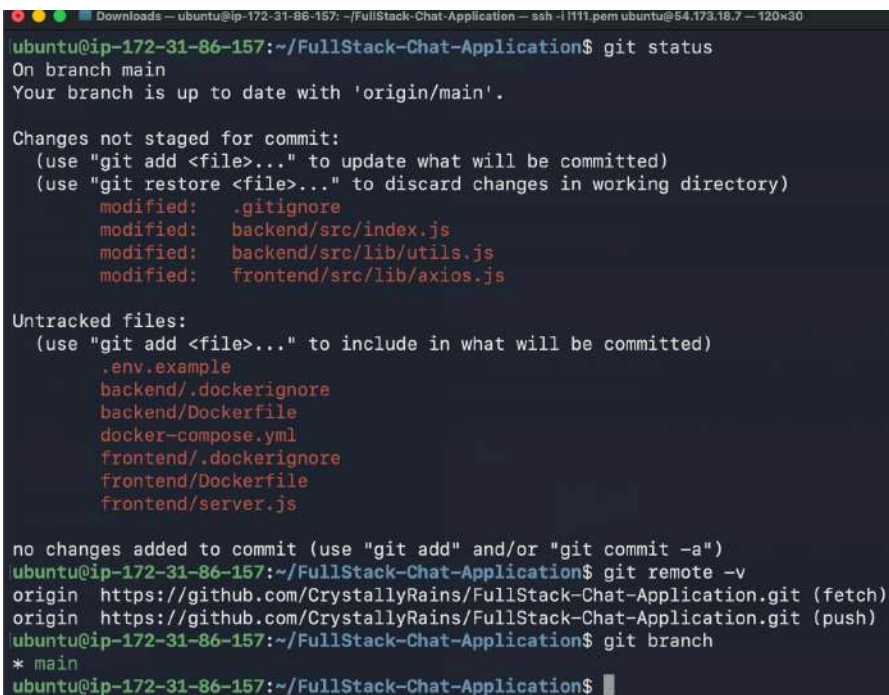
# 4. verify after logging back in
docker --version && docker compose version
```

Deploying the application

```
git clone https://github.com/<you>/<your-repo>.git
cd <your-repo>

# create the env file on the server, never commit it
cat > .env <<EOF
JWT_SECRET=<a long random string>
CLIENT_URL=http://<EC2_PUBLIC_IP>:5173
VITE_API_URL=http://<EC2_PUBLIC_IP>:8080/api
EOF

docker compose build
docker compose up -d
docker compose ps
```



The screenshot shows a terminal window with the following content:

```
ubuntu@ip-172-31-86-157: ~/FullStack-Chat-Application -- ssh -i i111.pem ubuntu@64.173.18.7 -- 120x30
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore
        modified:   backend/src/index.js
        modified:   backend/src/lib/utils.js
        modified:   frontend/src/lib/axios.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .env.example
        backend/.dockerignore
        backend/Dockerfile
        docker-compose.yml
        frontend/.dockerignore
        frontend/Dockerfile
        frontend/server.js

no changes added to commit (use "git add" and/or "git commit -a")
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ git remote -v
origin  https://github.com/CrystallyRains/FullStack-Chat-Application.git (fetch)
origin  https://github.com/CrystallyRains/FullStack-Chat-Application.git (push)
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ git branch
* main
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$
```



```

ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ git add .
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .env.example
    modified:   .gitignore
    new file:   backend/.dockerignore
    new file:   backend/Dockerfile
    modified:   backend/src/index.js
    modified:   backend/src/lib/utils.js
    new file:   docker-compose.yml
    new file:   frontend/.dockerignore
    new file:   frontend/Dockerfile
    new file:   frontend/server.js
    modified:   frontend/src/lib/axios.js

ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ git commit -m "Dockerize full stack chat application"
[main 6d38a70] Dockerize full stack chat application
  Committer: Ubuntu <ubuntu@ip-172-31-86-157.ec2.internal>
  Your name and email address were configured automatically based
  on your username and hostname. Please check that they are accurate.
  You can suppress this message by setting them explicitly. Run the
  following command and follow the instructions in your editor to edit
  your configuration file:

    git config --global --edit

```

```

git commit --amend --reset-author

11 files changed, 168 insertions(+), 8 deletions(-)
create mode 100644 .env.example
create mode 100644 backend/.dockerignore
create mode 100644 backend/Dockerfile
create mode 100644 docker-compose.yml
create mode 100644 frontend/.dockerignore
create mode 100644 frontend/Dockerfile
create mode 100644 frontend/server.js
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ git push origin main
Username for 'https://github.com': crystallymaine
Password for 'https://crystallymaine@github.com':
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 2 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (19/19), 2.97 MiB | 1.48 MiB/s, done.
Total 19 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (0/0), completed with 0 local objects.
To https://github.com:crystallymaine/Fullstack-Chat-Application.git
 +43ccc2..6d38a70 main -> main
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$

```

```

ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ ls
Readme.md backend docker-compose.yml frontend package-lock.json package.json
ubuntu@ip-172-31-86-157:~/FullStack-Chat-Application$ docker compose up -d
[+] Running 11/11
  ✓ database Pulled 11.2s
  ✓ 164558a2a8fa Pull complete 2.8s
  ✓ d544298cabd5 Pull complete 0.0s
  ✓ 77214491514e Pull complete 0.2s
  ✓ 9dad1a8753e0 Pull complete 0.2s
  ✓ 6bcb0cd9f68c Pull complete 0.1s
  ✓ 4afa172ff42c Pull complete 10.7s
  ✓ 586ba587042d Pull complete 3.0s
  ✓ ad1d6026847d Pull complete 0.2s
  ✓ a2b49326e024 Download complete 0.1s
  ✓ b6bc1923d416 Download complete 0.0s
WARN[0011] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 39.7s (29/29) FINISHED docker:default
=> [frontend internal] load build definition from Dockerfile 0.0s
=> == transferring dockerfile: 365B 0.0s
=> [backend internal] load build definition from Dockerfile 0.0s
=> == transferring dockerfile: 470B 0.0s
=> [backend internal] load metadata for dhi.io/node:24-alpine3.23 1.2s
=> [frontend internal] load metadata for dhi.io/node:24-alpine3.23-dev 1.3s
=> [backend auth] node:pull token for dhi.io 0.0s
=> [backend internal] load .dockerignore 0.0s
=> == transferring context: 2B 0.0s
=> [frontend internal] load .dockerignore 0.0s
=> == transferring context: 2B 0.0s
=> [backend builder 1/6] FROM dhi.io/node:24-alpine3.23-dev@sha256:609216ef29887f388b3be53ef980b33685ebbbbf36940 7.1s
=> == resolve dhi.io/node:24-alpine3.23-dev@sha256:609216ef29887f388b3be53ef980b33685ebbbbf36940cdd5462245230f28 0.1s

```

Verifying the deployment

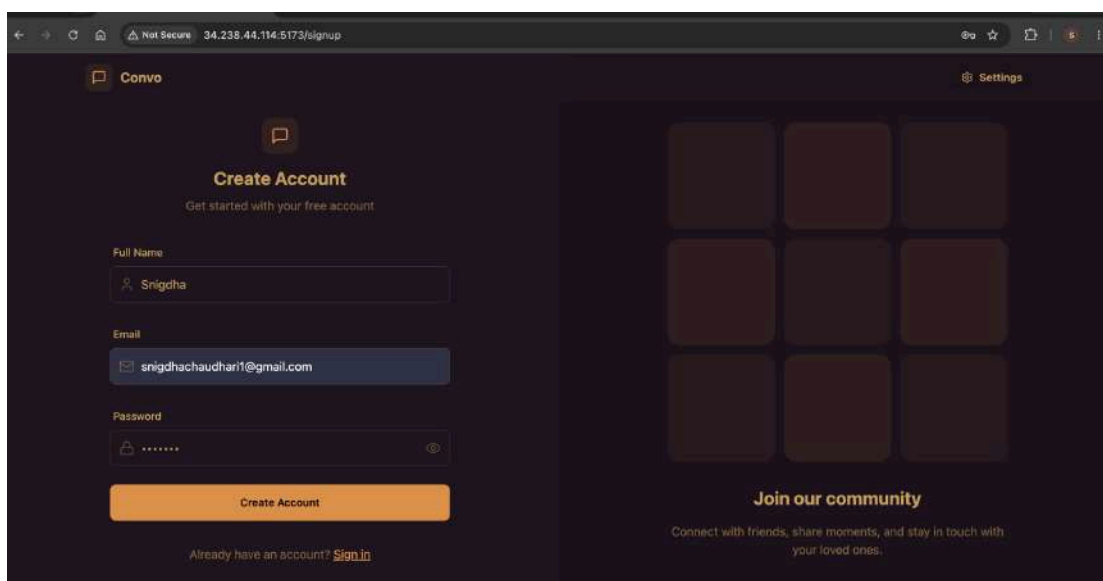
A container in the Up state proves that a process started. It does not prove that your application works.

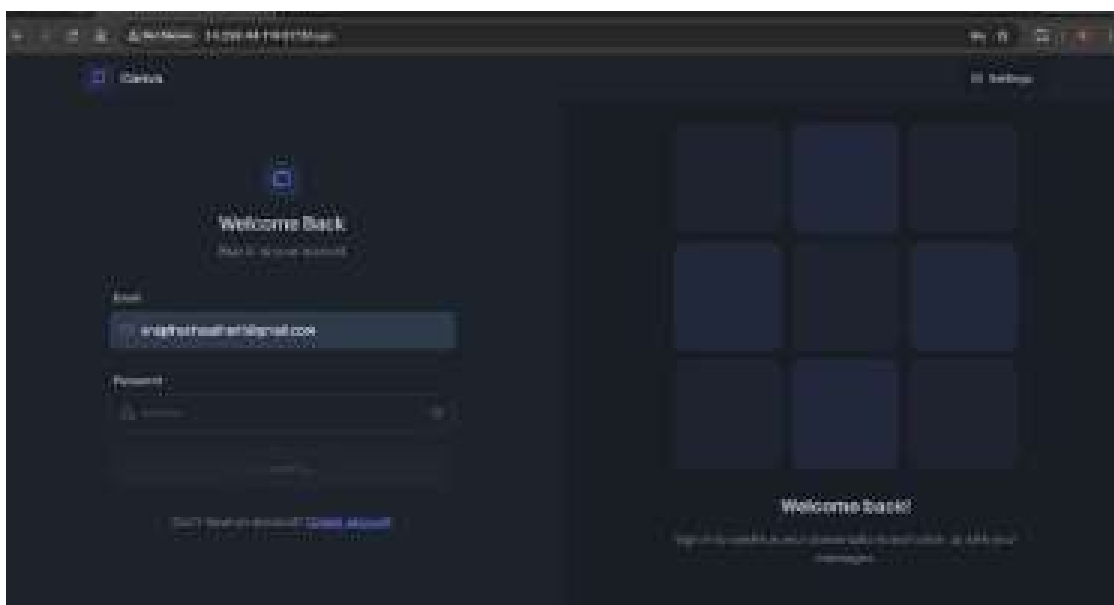
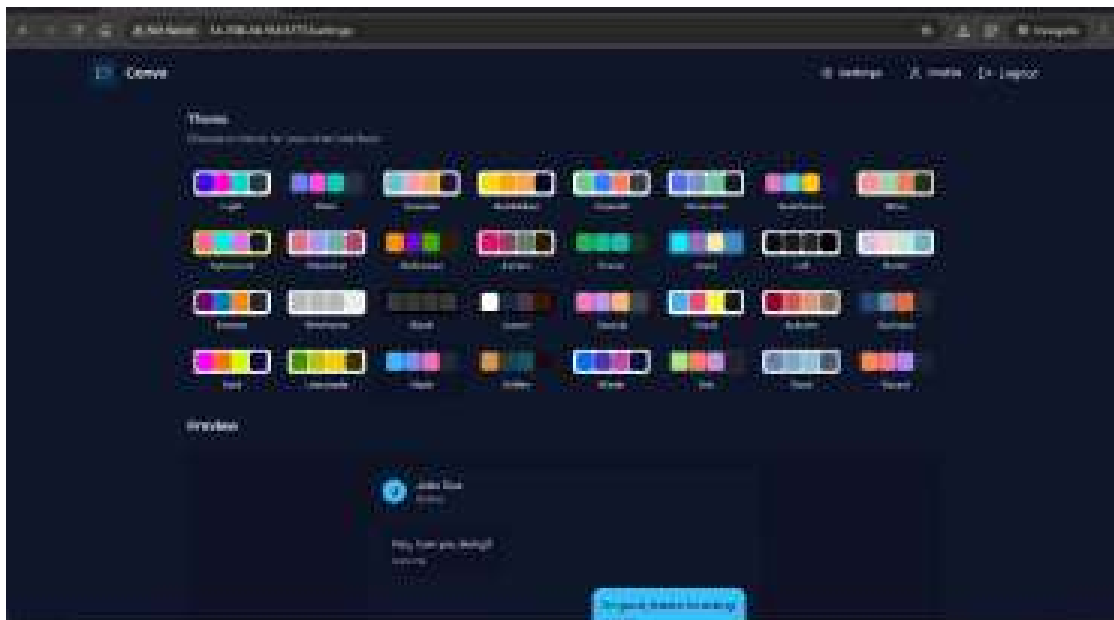
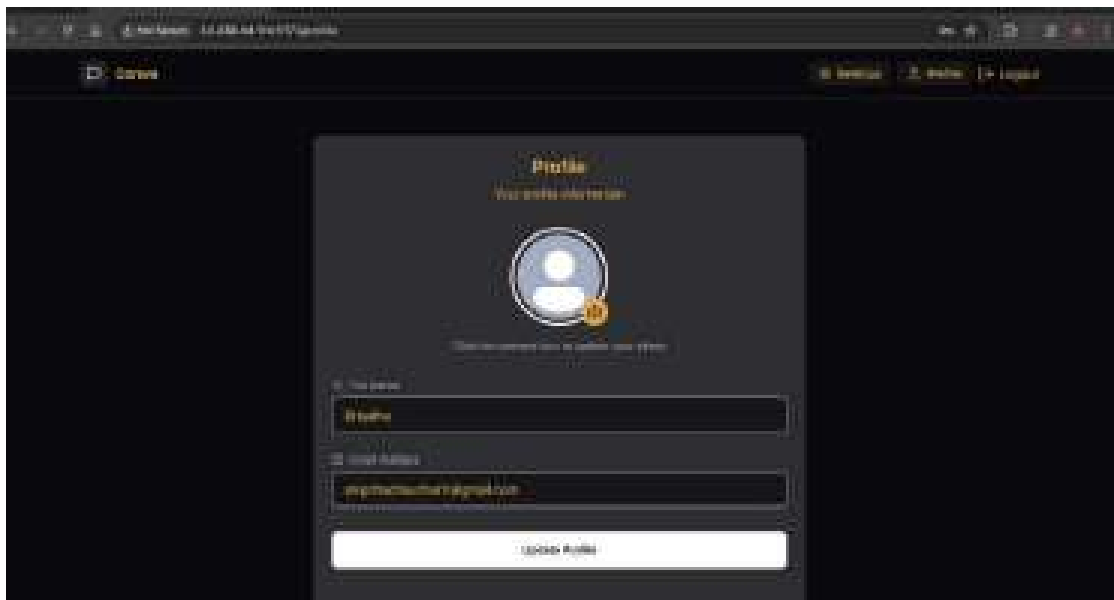
Check	How to test it	What it proves
Frontend serves	Open the public IP and port in a browser	Image built, nginx running, port published
API responds	<code>curl -i http://<ip>:8080/api/auth/check</code>	Backend container healthy and routed
Database connected	<code>docker compose logs backend</code>	Service name DNS and network are working
Signup works	Register a user in the UI	Write path all the way to MongoDB
Login works	Log in and reload the page	JWT issued and the cookie is being sent back
Two users chat	Second user in a private window	Websocket upgrade and message delivery
Data persists	<code>docker compose down && docker compose up -d</code>	The named volume is doing its job

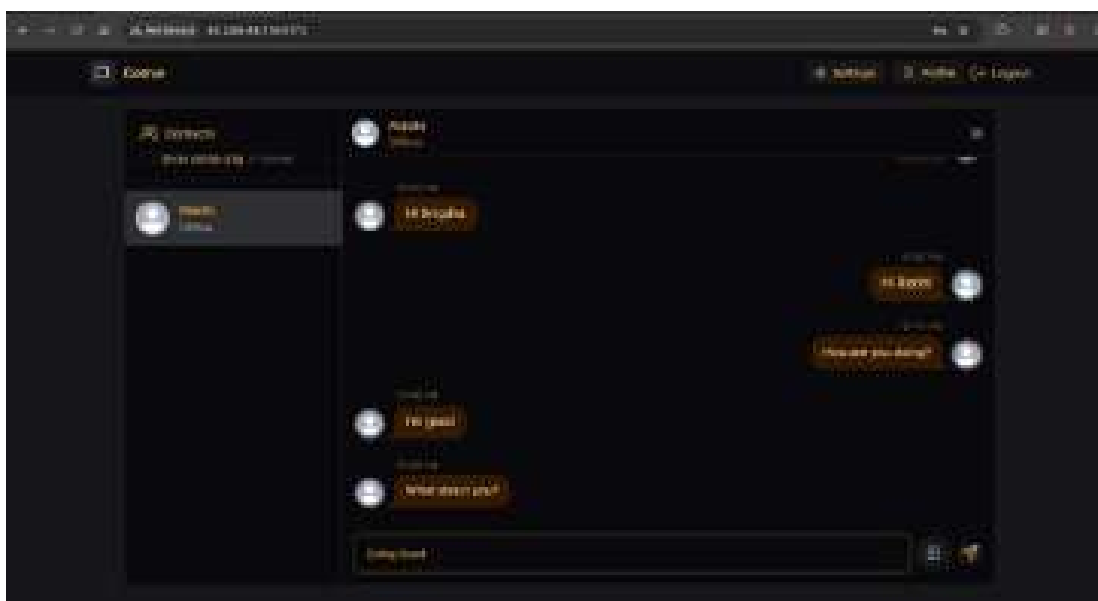
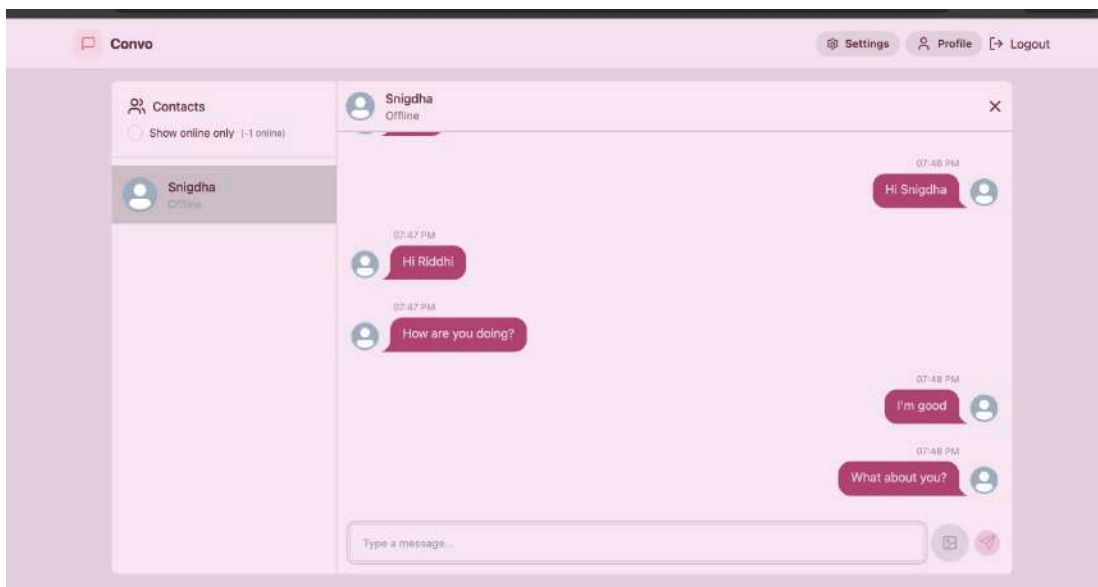
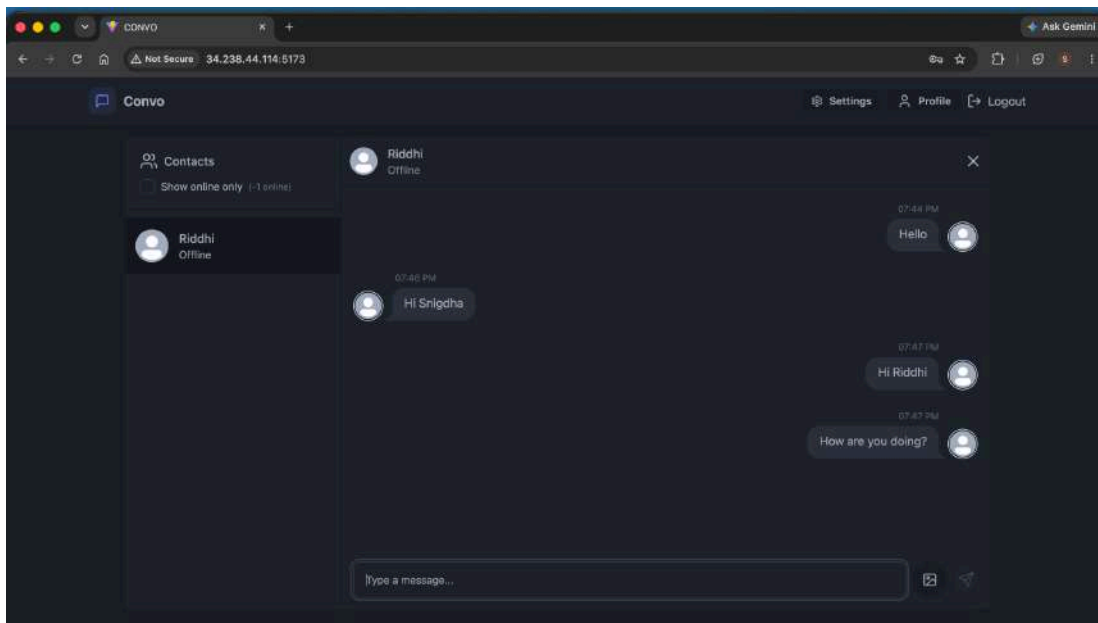
```

- [frontend] installing provenance for metadata file...
[+] Warning: 2/7
✓ backend Built 0.2s
✓ frontend Built 0.3s
✓ Network fullstack-chat-application_default Created 0.1s
✓ Network fullstack-chat-application_chatapp-net Created 0.1s
✓ Container fullstack-chat-application-backend-1 Started 0.3s
✓ Container fullstack-chat-application-database-1 Started 0.5s
✓ Container fullstack-chat-application-frontend-1 Started 0.7s
shuntu@ip-172-31-85-167:~/Fullstack-Chat-Applications$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
18c4c0819a0c   mongo:7.0-jammy                    "docker-entrypoint.sh"  18 seconds ago Up 18 seconds 27017/tcp
48a3f708a57c   fullstack-chat-application-database-1 "node src/index.js"    18 seconds ago Up 18 seconds 0.0.0.0:27018->27018/tcp
808c08030/1cp, [::]:8080->8080/tcp fullstack-chat-application-backend-1 "node src/index.js"    18 seconds ago Up 18 seconds 0.0.0.0:8080->8080/tcp
08c19c83e8ab   fullstack-chat-application-frontend "node server.js"       18 seconds ago Up 18 seconds 0.0.0.0:80->80/tcp
173c5e173/1cp, [::]:5173->5173/tcp fullstack-chat-application-frontend-1 "node server.js"       18 seconds ago Up 18 seconds 0.0.0.0:5173->5173/tcp
shuntu@ip-172-31-85-167:~/Fullstack-Chat-Applications$

```







Where this goes next

This setup is a solid learning deployment. It is not production. Knowing the difference is more valuable than pretending there is none.

Gap	What production looks like
Traffic is plain HTTP	TLS via a reverse proxy or load balancer, with certificates renewed automatically
The app is reached by IP	A domain name, DNS, and cookies that can safely use secure and sameSite settings
Secrets sit in a .env file	AWS Secrets Manager, SSM Parameter Store or a similar managed store
MongoDB runs as a container on the same host	A managed database with backups, monitoring and point in time recovery
Deploys are manual git pulls	A CI pipeline that builds, scans and pushes images to a registry
One instance is the whole system	Multiple replicas behind a load balancer, or a scheduler like ECS or Kubernetes
Logs live inside containers	Centralised logging and metrics, so an incident does not start with SSH
Images are never checked	Vulnerability scanning in the pipeline, with pinned base image versions

THE MENTAL MODEL TO KEEP

Docker gave you a repeatable answer to "what does this app need to run". Compose gave you a repeatable answer to "how do these pieces fit together". Everything on the list above is a repeatable answer to "what happens when this breaks at 3am". Same instinct, larger blast radius.

Command reference

The short list that covers most of the day to day.

Lifecycle

```
docker compose build          # build images
docker compose up -d         # start in background
docker compose up -d --build  # rebuild and start
docker compose ps            # status and port mappings
docker compose stop          # stop, keep containers
docker compose down          # remove containers + network
docker compose down -v       # also delete volumes (data loss)
```

Inspecting

```
docker compose logs -f backend  # follow one service
docker compose logs --tail=100  # recent output, all services
docker compose exec backend sh  # shell inside a container
docker compose exec backend printenv # what env did it actually get
docker stats                    # live CPU and memory per container
docker inspect chat-db         # full container detail
```

Volumes and cleanup

```
docker volume ls              # list volumes
docker volume inspect <name>  # mountpoint and usage
docker image ls               # images and their sizes
docker system df              # what is eating your disk
docker system prune -a        # remove unused images (careful)
```

From "it runs" to "I know why it runs."

That gap is the whole job. Every section in this guide is one step across it.